

■■ INFRASTRUCTURE FILES

Database Infrastructure Setup Scripts

Generated: August 27, 2025 at 01:39 PM

Location: database/infra/

Total Files: 5

■ TABLE OF CONTENTS

1. PostgreSQL Extensions Setup (01_extensions.sql)
2. Review System Tables (02_review_system.sql)
3. Embeddings and Vector Storage (03_embeddings_system.sql)
4. Event Tracking and Analytics (04_event_tracking.sql)
5. MLflow Integration Database Schema (05_mlflow_integration.sql)

■ *PostgreSQL Extensions Setup*

File: 01_extensions.sql

Description: Enables required PostgreSQL extensions (pgvector, uuid-oss)

```
-- =====
-- 01_extensions.sql - PostgreSQL Extensions Setup
-- =====
-- This script enables required PostgreSQL extensions
-- Run this first before any other scripts

-- Enable pgvector for embeddings
CREATE EXTENSION IF NOT EXISTS vector;

-- Enable UUID generation
CREATE EXTENSION IF NOT EXISTS "uuid-oss";

-- Verify extensions are installed
SELECT '■ Extensions setup complete!' as status;
SELECT extname, extversion FROM pg_extension WHERE extname IN ('vector', 'uuid-oss');
```

■ **Review System Tables**

File: 02_review_system.sql

Description: Creates review system tables for QA validation with comprehensive metrics

```
-- =====
-- 02_review_system.sql - Review System Tables
-- =====
-- This script creates the review system tables for QA validation
-- Run after 01_extensions.sql

-- Main review runs table with all metrics and audit fields
CREATE TABLE IF NOT EXISTS review_runs (
  id BIGSERIAL PRIMARY KEY,

  -- Identity & Organization
  reviewer TEXT,
  org TEXT,
  team TEXT,
  process TEXT,
  doc_id TEXT NOT NULL,
  review_date DATE NOT NULL,

  -- Audit & Provenance
  ruleset_hash TEXT NOT NULL,
  report_s3_uri TEXT NOT NULL,

  -- Basic Metrics
  runtime_sec INT,
  chunk_count INT,
  finding_count INT,
  tokens_input INT,
  tokens_output INT,
  model_calls JSONB,

  -- Enhanced Analytics
  pipeline_version TEXT,
  doc_pages INT,
  doc_bytes BIGINT,
  input_hash TEXT,
  prompt_hash TEXT,
  model_primary TEXT,
  latency_p50_ms INT,
  latency_p95_ms INT,
  cost_usd NUMERIC(10,4),
  error_count INT,
  timeout_count INT,
  retry_count INT,
  evidence_count INT,
  severity_counts JSONB,
  consensus_status TEXT,

  -- Timestamps
  created_at TIMESTAMPTZ DEFAULT now()
);
```

■ *Review System Tables (continued)*

```
-- Individual findings for each review run
CREATE TABLE IF NOT EXISTS review_findings (
  run_id BIGINT REFERENCES review_runs(id) ON DELETE CASCADE,
  rule_id TEXT NOT NULL,
  stage TEXT,
  severity TEXT,
  status TEXT,
  evidence JSONB,
  rule_source TEXT,
  PRIMARY KEY (run_id, rule_id)
);

-- Indexes for performance
CREATE INDEX IF NOT EXISTS review_runs_date ON review_runs(review_date);
CREATE INDEX IF NOT EXISTS review_runs_model ON review_runs(model_primary);
CREATE INDEX IF NOT EXISTS review_runs_ruleset ON review_runs(ruleset_hash);
CREATE INDEX IF NOT EXISTS review_runs_org ON review_runs(org);
CREATE INDEX IF NOT EXISTS review_runs_org_team ON review_runs(org, team);
CREATE INDEX IF NOT EXISTS review_runs_org_team_proc_date ON review_runs(org, team, process, review_date);
CREATE INDEX IF NOT EXISTS review_runs_reviewer ON review_runs(reviewer);

-- Verify setup
SELECT '■ Review system setup complete!' as status;
SELECT COUNT(*) as review_runs_count FROM review_runs;
SELECT COUNT(*) as review_findings_count FROM review_findings;
```

■ **Embeddings and Vector Storage**

File: 03_embeddings_system.sql

Description: Creates embeddings system with pgvector support and "Walmart-style" categorical filters

```
-- =====
-- 03_embeddings_system.sql - Embeddings and Vector Storage
-- =====
-- This script creates the embeddings system with pgvector support
-- Run after 02_review_system.sql

-- Section embeddings with "Walmart-style" categorical filters
CREATE TABLE IF NOT EXISTS section_gists (
  id BIGSERIAL PRIMARY KEY,

  -- Document Identity
  doc_id TEXT NOT NULL,
  section TEXT NOT NULL,
  chunk_ids TEXT[],

  -- "Walmart-style" Categorical Filters
  lifecycle TEXT,          -- "Development", "Validation", "Monitoring"
  stage TEXT,              -- "Gate1", "Gate2", "Gate3", "Deployment"
  control_owner TEXT,      -- Named owner/responsible party
  risk_tier TEXT,          -- "High", "Medium", "Low"
  doc_type TEXT,           -- "Plan", "Validation", "Monitoring", "Report"
  year INT,                -- Document year for temporal filtering
  tags TEXT[],             -- Flexible tagging: ["DQ", "lineage", "thresholds"]

  -- Embedding & Content
  embedding_model TEXT NOT NULL, -- "titan_v2_1024", "cohere_v3", etc.
  embedding vector(1024),        -- pgvector embedding (adjust dimension as needed)
  gist TEXT NOT NULL,            -- Compressed section summary (~120 tokens)

  -- Evidence & Provenance
  evidence_refs TEXT[],         -- S3 URIs to source PDF pages/sections
  s3_uri TEXT,                 -- URI to the .md artifact in S3
  content_hash TEXT,           -- SHA-1 of the gist content for deduplication

  -- Audit Trail
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now(),

  UNIQUE(doc_id, section)
);

-- Document metadata table
CREATE TABLE IF NOT EXISTS document_metadata (
  doc_id TEXT PRIMARY KEY,

  -- Document Properties
  title TEXT,
  doc_type TEXT,
  version TEXT,
  status TEXT,
```

■ *Embeddings and Vector Storage (continued)*

```
-- Categorical Organization
business_unit TEXT,
process_area TEXT,
risk_tier TEXT,
lifecycle_stage TEXT,

-- Document Stats
total_pages INT,
total_sections INT,
total_chunks INT,
file_size_bytes BIGINT,

-- S3 Storage
s3_uri TEXT,
processed_prefix TEXT,

-- Timestamps
document_date DATE,
ingested_at TIMESTAMPTZ DEFAULT now(),
last_processed TIMESTAMPTZ
);

-- Individual chunk embeddings
CREATE TABLE IF NOT EXISTS document_chunks (
  id BIGSERIAL PRIMARY KEY,
  doc_id TEXT NOT NULL,
  chunk_id TEXT NOT NULL,
  section TEXT,
  page_num INT,
  chunk_text TEXT NOT NULL,
  embedding_model TEXT NOT NULL,
  embedding vector(1024),
  char_count INT,
  token_estimate INT,
  s3_uri TEXT,
  content_hash TEXT,
  created_at TIMESTAMPTZ DEFAULT now(),
  UNIQUE(doc_id, chunk_id)
);

-- Indexes for performance
CREATE INDEX IF NOT EXISTS section_gists_doc_id ON section_gists(doc_id);
CREATE INDEX IF NOT EXISTS section_gists_lifecycle ON section_gists(lifecycle);
CREATE INDEX IF NOT EXISTS section_gists_stage ON section_gists(stage);
CREATE INDEX IF NOT EXISTS section_gists_risk_tier ON section_gists(risk_tier);
CREATE INDEX IF NOT EXISTS section_gists_year ON section_gists(year);
CREATE INDEX IF NOT EXISTS section_gists_embedding_model ON section_gists(embedding_model);

CREATE INDEX IF NOT EXISTS document_chunks_doc_id ON document_chunks(doc_id);
```

■ *Embeddings and Vector Storage (continued)*

```
CREATE INDEX IF NOT EXISTS document_chunks_section ON document_chunks(section);
CREATE INDEX IF NOT EXISTS document_chunks_embedding_model ON document_chunks(embedding_model);

-- Vector similarity search indexes (HNSW for fast approximate search)
CREATE INDEX IF NOT EXISTS section_gists_embedding_idx ON section_gists USING hnsw (embedding_vector_cos);
CREATE INDEX IF NOT EXISTS document_chunks_embedding_idx ON document_chunks USING hnsw (embedding_vector_cos);

-- Verify setup
SELECT '■ Embeddings system setup complete!' as status;
SELECT COUNT(*) as section_gists_count FROM section_gists;
SELECT COUNT(*) as document_metadata_count FROM document_metadata;
SELECT COUNT(*) as document_chunks_count FROM document_chunks;
```

■ *Event Tracking and Analytics*

File: 04_event_tracking.sql

Description: Creates event tracking tables for analytics and monitoring

```
-- =====
-- 04_event_tracking.sql - Event Tracking and Analytics
-- =====
-- This script creates event tracking tables for analytics
-- Run after 03_embeddings_system.sql

-- Raw events for detailed tracking
CREATE TABLE IF NOT EXISTS events_raw (
  id BIGSERIAL PRIMARY KEY,
  ts TIMESTAMPTZ NOT NULL DEFAULT now(),
  user_id TEXT,
  org TEXT,
  team TEXT,
  process TEXT,
  event TEXT NOT NULL,    -- e.g., auth_login, session_start, page_view, config_change,
                           -- validation_error, app_error, scoring_update, run_started, run_finished
  payload JSONB           -- free-form data, avoid PII
);

-- Daily aggregated metrics for fast reporting
CREATE TABLE IF NOT EXISTS events_daily (
  day DATE NOT NULL,
  org TEXT,
  team TEXT,
  process TEXT,
  metric TEXT NOT NULL,   -- e.g., active_users, page_views, runs, burgers, errors
  value BIGINT NOT NULL,
  PRIMARY KEY(day, org, team, process, metric)
);

-- Indexes for performance
CREATE INDEX IF NOT EXISTS events_raw_ts ON events_raw(ts);
CREATE INDEX IF NOT EXISTS events_raw_event ON events_raw(event);
CREATE INDEX IF NOT EXISTS events_raw_user ON events_raw(user_id);
CREATE INDEX IF NOT EXISTS events_raw_org_team ON events_raw(org, team);

CREATE INDEX IF NOT EXISTS events_daily_day ON events_daily(day);
CREATE INDEX IF NOT EXISTS events_daily_metric ON events_daily(metric);
CREATE INDEX IF NOT EXISTS events_daily_org_team ON events_daily(org, team);

-- Verify setup
SELECT '■ Event tracking setup complete!' as status;
SELECT COUNT(*) as events_raw_count FROM events_raw;
SELECT COUNT(*) as events_daily_count FROM events_daily;
```

■ **MLflow Integration Database Schema**

File: 05_mlflow_integration.sql

Description: Creates database schema for MLflow integration with Unified LLM Gateway system

```
-- =====
-- MLflow Integration Database Schema
-- =====
-- This script creates the database schema for MLflow integration
-- with the Unified LLM Gateway system
--
-- Run this after setting up PostgreSQL and before starting MLflow server
-- =====
-- 1. MLflow Core Tables (Required by MLflow)
-- =====

-- Experiments table
CREATE TABLE IF NOT EXISTS experiments (
    experiment_id INTEGER PRIMARY KEY,
    name VARCHAR(256) NOT NULL,
    artifact_location VARCHAR(256),
    lifecycle_stage VARCHAR(32)
);

-- Runs table
CREATE TABLE IF NOT EXISTS runs (
    run_uuid VARCHAR(32) PRIMARY KEY,
    name VARCHAR(250),
    source_type VARCHAR(32),
    source_name VARCHAR(500),
    entry_point_name VARCHAR(200),
    user_id VARCHAR(256),
    status VARCHAR(9),
    start_time BIGINT,
    end_time BIGINT,
    source_version VARCHAR(50),
    lifecycle_stage VARCHAR(32),
    artifact_uri VARCHAR(200),
    experiment_id INTEGER,
    FOREIGN KEY (experiment_id) REFERENCES experiments(experiment_id)
);

-- Parameters table
CREATE TABLE IF NOT EXISTS params (
    key VARCHAR(250) NOT NULL,
    value VARCHAR(250) NOT NULL,
    run_uuid VARCHAR(32) NOT NULL,
    PRIMARY KEY (key, run_uuid),
    FOREIGN KEY (run_uuid) REFERENCES runs(run_uuid) ON DELETE CASCADE
);

-- Metrics table
CREATE TABLE IF NOT EXISTS metrics (
    key VARCHAR(250) NOT NULL,
```

■ *MLflow Integration Database Schema (continued)*

```
        value DOUBLE PRECISION NOT NULL,
        timestamp BIGINT NOT NULL,
        run_uuid VARCHAR(32) NOT NULL,
        step BIGINT DEFAULT 0,
        PRIMARY KEY (key, run_uuid, timestamp, step),
        FOREIGN KEY (run_uuid) REFERENCES runs(run_uuid) ON DELETE CASCADE
    );

-- Tags table
CREATE TABLE IF NOT EXISTS tags (
    key VARCHAR(250) NOT NULL,
    value VARCHAR(250) NOT NULL,
    run_uuid VARCHAR(32) NOT NULL,
    PRIMARY KEY (key, run_uuid),
    FOREIGN KEY (run_uuid) REFERENCES runs(run_uuid) ON DELETE CASCADE
);

-- =====
-- 2. Unified LLM Gateway Integration Tables
-- =====

-- LLM Gateway Configuration
CREATE TABLE IF NOT EXISTS llm_gateway_config (
    id SERIAL PRIMARY KEY,
    gateway_name VARCHAR(100) NOT NULL UNIQUE,
    gateway_type VARCHAR(50) NOT NULL, -- 'local', 'remote', 'unified'
    base_url VARCHAR(500) NOT NULL,
    api_key VARCHAR(500), -- Encrypted
    model_preferences JSONB,
    is_active BOOLEAN DEFAULT true,
    created_at TIMESTAMPTZ DEFAULT now(),
    updated_at TIMESTAMPTZ DEFAULT now()
);

-- LLM Call History (Enhanced tracking)
CREATE TABLE IF NOT EXISTS llm_call_history (
    id BIGSERIAL PRIMARY KEY,
    run_uuid VARCHAR(32) REFERENCES runs(run_uuid) ON DELETE CASCADE,
    agent_name VARCHAR(100) NOT NULL,
    task_type VARCHAR(100) NOT NULL,
    model_name VARCHAR(100) NOT NULL,
    gateway_type VARCHAR(50) NOT NULL, -- 'local', 'remote'
    input_tokens INTEGER,
    output_tokens INTEGER,
    total_tokens INTEGER,
    cost_usd NUMERIC(10,6),
    response_time_ms INTEGER,
    success BOOLEAN DEFAULT true,
    error_message TEXT,
    prompt_hash VARCHAR(64), -- SHA-256 hash of prompt
```

■ *MLflow Integration Database Schema (continued)*

```
        response_hash VARCHAR(64), -- SHA-256 hash of response
        created_at TIMESTAMPTZ DEFAULT now()
    );

-- Batch Processing Tracking
CREATE TABLE IF NOT EXISTS llm_batch_processing (
    batch_id VARCHAR(100) PRIMARY KEY,
    experiment_name VARCHAR(256) NOT NULL,
    total_calls INTEGER DEFAULT 0,
    successful_calls INTEGER DEFAULT 0,
    failed_calls INTEGER DEFAULT 0,
    total_tokens INTEGER DEFAULT 0,
    total_cost_usd NUMERIC(10,6) DEFAULT 0,
    avg_response_time_ms INTEGER,
    budget_limit_usd NUMERIC(10,6),
    status VARCHAR(50) DEFAULT 'running', -- 'running', 'completed', 'failed'
    started_at TIMESTAMPTZ DEFAULT now(),
    completed_at TIMESTAMPTZ,
    created_by VARCHAR(100)
);

-- =====
-- 3. Enhanced Document Processing Tables
-- =====

-- Document Processing Sessions
CREATE TABLE IF NOT EXISTS document_processing_sessions (
    session_id VARCHAR(100) PRIMARY KEY,
    batch_id VARCHAR(100) REFERENCES llm_batch_processing(batch_id),
    experiment_name VARCHAR(256) NOT NULL,
    input_directory VARCHAR(500),
    output_directory VARCHAR(500),
    total_documents INTEGER DEFAULT 0,
    processed_documents INTEGER DEFAULT 0,
    failed_documents INTEGER DEFAULT 0,
    status VARCHAR(50) DEFAULT 'running',
    started_at TIMESTAMPTZ DEFAULT now(),
    completed_at TIMESTAMPTZ,
    metadata JSONB
);

-- Document Classification Results
CREATE TABLE IF NOT EXISTS document_classifications (
    id BIGSERIAL PRIMARY KEY,
    session_id VARCHAR(100) REFERENCES document_processing_sessions(session_id),
    run_uuid VARCHAR(32) REFERENCES runs(run_uuid),
    document_filename VARCHAR(500) NOT NULL,
    document_hash VARCHAR(64),
    classification VARCHAR(100),
    confidence_score NUMERIC(5,2),
```

■ *MLflow Integration Database Schema (continued)*

```
        key_themes TEXT[],
        document_purpose TEXT,
        processing_time_ms INTEGER,
        created_at TIMESTAMPTZ DEFAULT now()
    );

-- Document Metadata Extraction
CREATE TABLE IF NOT EXISTS document_metadata (
    id BIGSERIAL PRIMARY KEY,
    session_id VARCHAR(100) REFERENCES document_processing_sessions(session_id),
    run_uuid VARCHAR(32) REFERENCES runs(run_uuid),
    document_filename VARCHAR(500) NOT NULL,
    title VARCHAR(500),
    authors TEXT[],
    document_date DATE,
    version VARCHAR(50),
    organization VARCHAR(200),
    document_type VARCHAR(100),
    key_topics TEXT[],
    summary TEXT,
    created_at TIMESTAMPTZ DEFAULT now()
);

-- =====
-- 4. Performance and Analytics Tables
-- =====

-- Gateway Performance Metrics
CREATE TABLE IF NOT EXISTS gateway_performance (
    id BIGSERIAL PRIMARY KEY,
    gateway_type VARCHAR(50) NOT NULL,
    model_name VARCHAR(100) NOT NULL,
    date_hour TIMESTAMPTZ NOT NULL,
    total_calls INTEGER DEFAULT 0,
    successful_calls INTEGER DEFAULT 0,
    failed_calls INTEGER DEFAULT 0,
    total_tokens INTEGER DEFAULT 0,
    total_cost_usd NUMERIC(10,6) DEFAULT 0,
    avg_response_time_ms INTEGER,
    p50_response_time_ms INTEGER,
    p95_response_time_ms INTEGER,
    p99_response_time_ms INTEGER,
    created_at TIMESTAMPTZ DEFAULT now(),
    UNIQUE(gateway_type, model_name, date_hour)
);

-- Cost Tracking by Experiment
CREATE TABLE IF NOT EXISTS experiment_cost_tracking (
    id BIGSERIAL PRIMARY KEY,
    experiment_name VARCHAR(256) NOT NULL,
```

■ *MLflow Integration Database Schema (continued)*

```
        date DATE NOT NULL,
        total_calls INTEGER DEFAULT 0,
        total_tokens INTEGER DEFAULT 0,
        total_cost_usd NUMERIC(10,6) DEFAULT 0,
        avg_cost_per_call NUMERIC(10,6),
        created_at TIMESTAMPTZ DEFAULT now(),
        UNIQUE(experiment_name, date)
    );

-- =====
-- 5. Indexes for Performance
-- =====

-- MLflow indexes
CREATE INDEX IF NOT EXISTS idx_runs_experiment_id ON runs(experiment_id);
CREATE INDEX IF NOT EXISTS idx_runs_user_id ON runs(user_id);
CREATE INDEX IF NOT EXISTS idx_runs_status ON runs(status);
CREATE INDEX IF NOT EXISTS idx_runs_start_time ON runs(start_time);
CREATE INDEX IF NOT EXISTS idx_metrics_run_uuid ON metrics(run_uuid);
CREATE INDEX IF NOT EXISTS idx_metrics_key ON metrics(key);
CREATE INDEX IF NOT EXISTS idx_params_run_uuid ON params(run_uuid);
CREATE INDEX IF NOT EXISTS idx_tags_run_uuid ON tags(run_uuid);

-- LLM Gateway indexes
CREATE INDEX IF NOT EXISTS idx_llm_call_history_run_uuid ON llm_call_history(run_uuid);
CREATE INDEX IF NOT EXISTS idx_llm_call_history_agent_name ON llm_call_history(agent_name);
CREATE INDEX IF NOT EXISTS idx_llm_call_history_task_type ON llm_call_history(task_type);
CREATE INDEX IF NOT EXISTS idx_llm_call_history_gateway_type ON llm_call_history(gateway_type);
CREATE INDEX IF NOT EXISTS idx_llm_call_history_created_at ON llm_call_history(created_at);
CREATE INDEX IF NOT EXISTS idx_llm_call_history_success ON llm_call_history(success);

-- Document processing indexes
CREATE INDEX IF NOT EXISTS idx_document_classifications_session_id ON document_classifications(session_id);
CREATE INDEX IF NOT EXISTS idx_document_classifications_classification ON document_classifications(classification);
CREATE INDEX IF NOT EXISTS idx_document_metadata_session_id ON document_metadata(session_id);
CREATE INDEX IF NOT EXISTS idx_document_metadata_document_type ON document_metadata(document_type);

-- Performance indexes
CREATE INDEX IF NOT EXISTS idx_gateway_performance_date_hour ON gateway_performance(date_hour);
CREATE INDEX IF NOT EXISTS idx_gateway_performance_gateway_model ON gateway_performance(gateway_type, model);
CREATE INDEX IF NOT EXISTS idx_experiment_cost_tracking_date ON experiment_cost_tracking(date);
CREATE INDEX IF NOT EXISTS idx_experiment_cost_tracking_experiment ON experiment_cost_tracking(experiment_name);

-- =====
-- 6. Views for Analytics
-- =====

-- LLM Gateway Performance Summary
CREATE OR REPLACE VIEW llm_gateway_performance_summary AS
SELECT
```

■ *MLflow Integration Database Schema (continued)*

```
gateway_type,
model_name,
COUNT(*) as total_calls,
SUM(CASE WHEN success THEN 1 ELSE 0 END) as successful_calls,
SUM(CASE WHEN NOT success THEN 1 ELSE 0 END) as failed_calls,
ROUND(SUM(CASE WHEN success THEN 1 ELSE 0 END) * 100.0 / COUNT(*), 2) as success_rate,
SUM(total_tokens) as total_tokens,
SUM(cost_usd) as total_cost_usd,
ROUND(AVG(response_time_ms), 2) as avg_response_time_ms,
ROUND(PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY response_time_ms), 2) as p50_response_time_ms,
ROUND(PERCENTILE_CONT(0.95) WITHIN GROUP (ORDER BY response_time_ms), 2) as p95_response_time_ms
FROM llm_call_history
WHERE created_at >= NOW() - INTERVAL '24 hours'
GROUP BY gateway_type, model_name
ORDER BY total_calls DESC;

-- Experiment Cost Summary
CREATE OR REPLACE VIEW experiment_cost_summary AS
SELECT
    e.name as experiment_name,
    COUNT(r.run_uuid) as total_runs,
    SUM(lch.total_tokens) as total_tokens,
    SUM(lch.cost_usd) as total_cost_usd,
    ROUND(AVG(lch.cost_usd), 6) as avg_cost_per_call,
    MIN(r.start_time) as first_run,
    MAX(r.start_time) as last_run
FROM experiments e
LEFT JOIN runs r ON e.experiment_id = r.experiment_id
LEFT JOIN llm_call_history lch ON r.run_uuid = lch.run_uuid
WHERE r.start_time >= EXTRACT(EPOCH FROM (NOW() - INTERVAL '7 days')) * 1000
GROUP BY e.experiment_id, e.name
ORDER BY total_cost_usd DESC;

-- Document Processing Summary
CREATE OR REPLACE VIEW document_processing_summary AS
SELECT
    dps.session_id,
    dps.experiment_name,
    dps.total_documents,
    dps.processed_documents,
    dps.failed_documents,
    ROUND(dps.processed_documents * 100.0 / dps.total_documents, 2) as success_rate,
    COUNT(DISTINCT dc.classification) as unique_classifications,
    COUNT(DISTINCT dm.document_type) as unique_document_types,
    dps.started_at,
    dps.completed_at,
    EXTRACT(EPOCH FROM (dps.completed_at - dps.started_at)) as processing_time_seconds
FROM document_processing_sessions dps
LEFT JOIN document_classifications dc ON dps.session_id = dc.session_id
LEFT JOIN document_metadata dm ON dps.session_id = dm.session_id
```

■ *MLflow Integration Database Schema (continued)*

```
GROUP BY dps.session_id, dps.experiment_name, dps.total_documents, dps.processed_documents,
        dps.failed_documents, dps.started_at, dps.completed_at
ORDER BY dps.started_at DESC;

-- =====
-- 7. Functions for Data Management
-- =====

-- Function to update gateway performance metrics
CREATE OR REPLACE FUNCTION update_gateway_performance()
RETURNS void AS $$
BEGIN
    INSERT INTO gateway_performance (
        gateway_type, model_name, date_hour, total_calls, successful_calls,
        failed_calls, total_tokens, total_cost_usd, avg_response_time_ms,
        p50_response_time_ms, p95_response_time_ms, p99_response_time_ms
    )
    SELECT
        gateway_type,
        model_name,
        DATE_TRUNC('hour', created_at) as date_hour,
        COUNT(*) as total_calls,
        SUM(CASE WHEN success THEN 1 ELSE 0 END) as successful_calls,
        SUM(CASE WHEN NOT success THEN 1 ELSE 0 END) as failed_calls,
        SUM(total_tokens) as total_tokens,
        SUM(cost_usd) as total_cost_usd,
        ROUND(AVG(response_time_ms)) as avg_response_time_ms,
        ROUND(PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY response_time_ms)) as p50_response_time_ms,
        ROUND(PERCENTILE_CONT(0.95) WITHIN GROUP (ORDER BY response_time_ms)) as p95_response_time_ms,
        ROUND(PERCENTILE_CONT(0.99) WITHIN GROUP (ORDER BY response_time_ms)) as p99_response_time_ms
    FROM llm_call_history
    WHERE created_at >= NOW() - INTERVAL '1 hour'
    GROUP BY gateway_type, model_name, DATE_TRUNC('hour', created_at)
    ON CONFLICT (gateway_type, model_name, date_hour)
    DO UPDATE SET
        total_calls = EXCLUDED.total_calls,
        successful_calls = EXCLUDED.successful_calls,
        failed_calls = EXCLUDED.failed_calls,
        total_tokens = EXCLUDED.total_tokens,
        total_cost_usd = EXCLUDED.total_cost_usd,
        avg_response_time_ms = EXCLUDED.avg_response_time_ms,
        p50_response_time_ms = EXCLUDED.p50_response_time_ms,
        p95_response_time_ms = EXCLUDED.p95_response_time_ms,
        p99_response_time_ms = EXCLUDED.p99_response_time_ms;
END;
$$ LANGUAGE plpgsql;

-- Function to update experiment cost tracking
CREATE OR REPLACE FUNCTION update_experiment_cost_tracking()
RETURNS void AS $$
```

■ *MLflow Integration Database Schema (continued)*

```
BEGIN
  INSERT INTO experiment_cost_tracking (
    experiment_name, date, total_calls, total_tokens, total_cost_usd, avg_cost_per_call
  )
  SELECT
    e.name as experiment_name,
    DATE(FROM_UNIXTIME(r.start_time / 1000)) as date,
    COUNT(lch.id) as total_calls,
    SUM(lch.total_tokens) as total_tokens,
    SUM(lch.cost_usd) as total_cost_usd,
    ROUND(AVG(lch.cost_usd), 6) as avg_cost_per_call
  FROM experiments e
  JOIN runs r ON e.experiment_id = r.experiment_id
  JOIN llm_call_history lch ON r.run_uuid = lch.run_uuid
  WHERE DATE(FROM_UNIXTIME(r.start_time / 1000)) = CURRENT_DATE
  GROUP BY e.name, DATE(FROM_UNIXTIME(r.start_time / 1000))
  ON CONFLICT (experiment_name, date)
  DO UPDATE SET
    total_calls = EXCLUDED.total_calls,
    total_tokens = EXCLUDED.total_tokens,
    total_cost_usd = EXCLUDED.total_cost_usd,
    avg_cost_per_call = EXCLUDED.avg_cost_per_call;
END;
$$ LANGUAGE plpgsql;

-- =====
-- 8. Triggers for Automatic Updates
-- =====

-- Trigger to update gateway performance when new calls are added
CREATE OR REPLACE FUNCTION trigger_update_gateway_performance()
RETURNS TRIGGER AS $$
BEGIN
  PERFORM update_gateway_performance();
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER update_gateway_performance_trigger
AFTER INSERT ON llm_call_history
FOR EACH ROW
EXECUTE FUNCTION trigger_update_gateway_performance();

-- Trigger to update experiment cost tracking when new calls are added
CREATE OR REPLACE FUNCTION trigger_update_experiment_cost()
RETURNS TRIGGER AS $$
BEGIN
  PERFORM update_experiment_cost_tracking();
  RETURN NEW;
END;
```

■ *MLflow Integration Database Schema (continued)*

```
$$ LANGUAGE plpgsql;

CREATE TRIGGER update_experiment_cost_trigger
    AFTER INSERT ON llm_call_history
    FOR EACH ROW
    EXECUTE FUNCTION trigger_update_experiment_cost();

-- =====
-- 9. Initial Data
-- =====

-- Insert default gateway configurations
INSERT INTO llm_gateway_config (gateway_name, gateway_type, base_url, model_preferences) VALUES
('local-zllm', 'local', 'http://localhost:11434', '{"default": "llama2", "classification": "gpt-4", "sum
('remote-openai', 'remote', 'https://api.openai.com/v1', '{"default": "gpt-3.5-turbo", "classification":
('unified-gateway', 'unified', 'http://localhost:5000', '{"default": "auto", "classification": "auto", "
ON CONFLICT (gateway_name) DO NOTHING;

-- =====
-- 10. Verification Queries
-- =====

-- Verify MLflow tables
SELECT 'MLflow Core Tables' as category, COUNT(*) as table_count FROM information_schema.tables
WHERE table_schema = 'public' AND table_name IN ('experiments', 'runs', 'params', 'metrics', 'tags');

-- Verify LLM Gateway tables
SELECT 'LLM Gateway Tables' as category, COUNT(*) as table_count FROM information_schema.tables
WHERE table_schema = 'public' AND table_name IN ('llm_gateway_config', 'llm_call_history', 'llm_batch_pr

-- Verify Document Processing tables
SELECT 'Document Processing Tables' as category, COUNT(*) as table_count FROM information_schema.tables
WHERE table_schema = 'public' AND table_name IN ('document_processing_sessions', 'document_classification

-- Verify Performance tables
SELECT 'Performance Tables' as category, COUNT(*) as table_count FROM information_schema.tables
WHERE table_schema = 'public' AND table_name IN ('gateway_performance', 'experiment_cost_tracking');

-- Show all created tables
SELECT
    table_name,
    pg_size_pretty(pg_total_relation_size(table_name::regclass)) as size
FROM information_schema.tables
WHERE table_schema = 'public'
ORDER BY pg_total_relation_size(table_name::regclass) DESC;

-- =====
-- SUCCESS MESSAGE
-- =====
```

■ *MLflow Integration Database Schema (continued)*

```
SELECT '■ MLflow Integration Database Schema created successfully!' as status;  
SELECT '■ Tables created: ' || COUNT(*) as table_count FROM information_schema.tables WHERE table_schema = 'mlflow';  
SELECT '■ Ready to connect Unified LLM Gateway to MLflow!' as next_step;
```